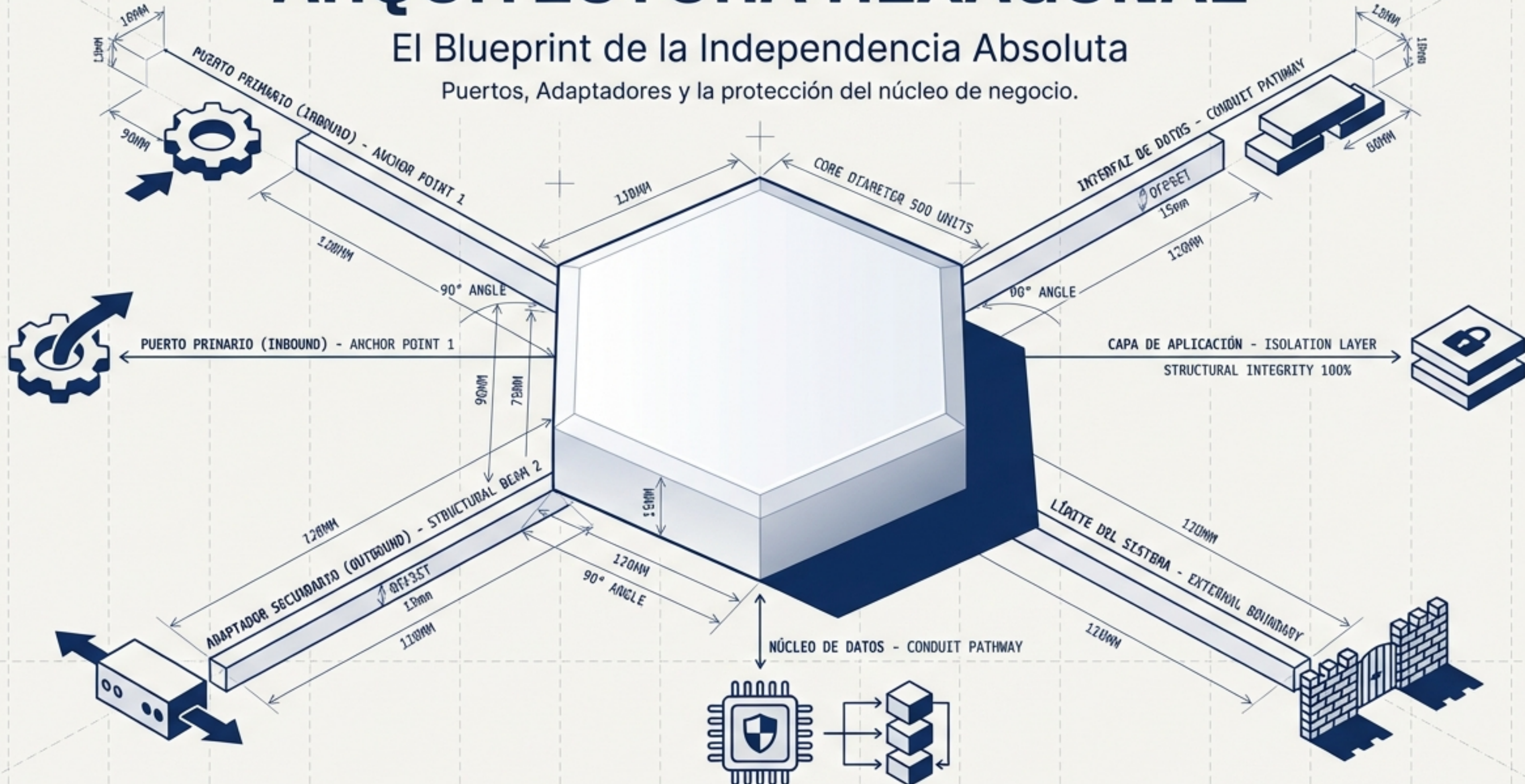


ARQUITECTURA HEXAGONAL

El Blueprint de la Independencia Absoluta

Puertos, Adaptadores y la protección del núcleo de negocio.



El núcleo no sabe con quién habla

La premisa es simple: aislar el corazón de tu aplicación para que no dependa de nada externo. Ni de la base de datos, ni de la interfaz web, ni de servicios de terceros.



El núcleo habla a través de contratos; el exterior se conecta mediante implementaciones concretas.

La Anatomía de las Tres Capas

1. Núcleo de la Aplicación (Application Core)

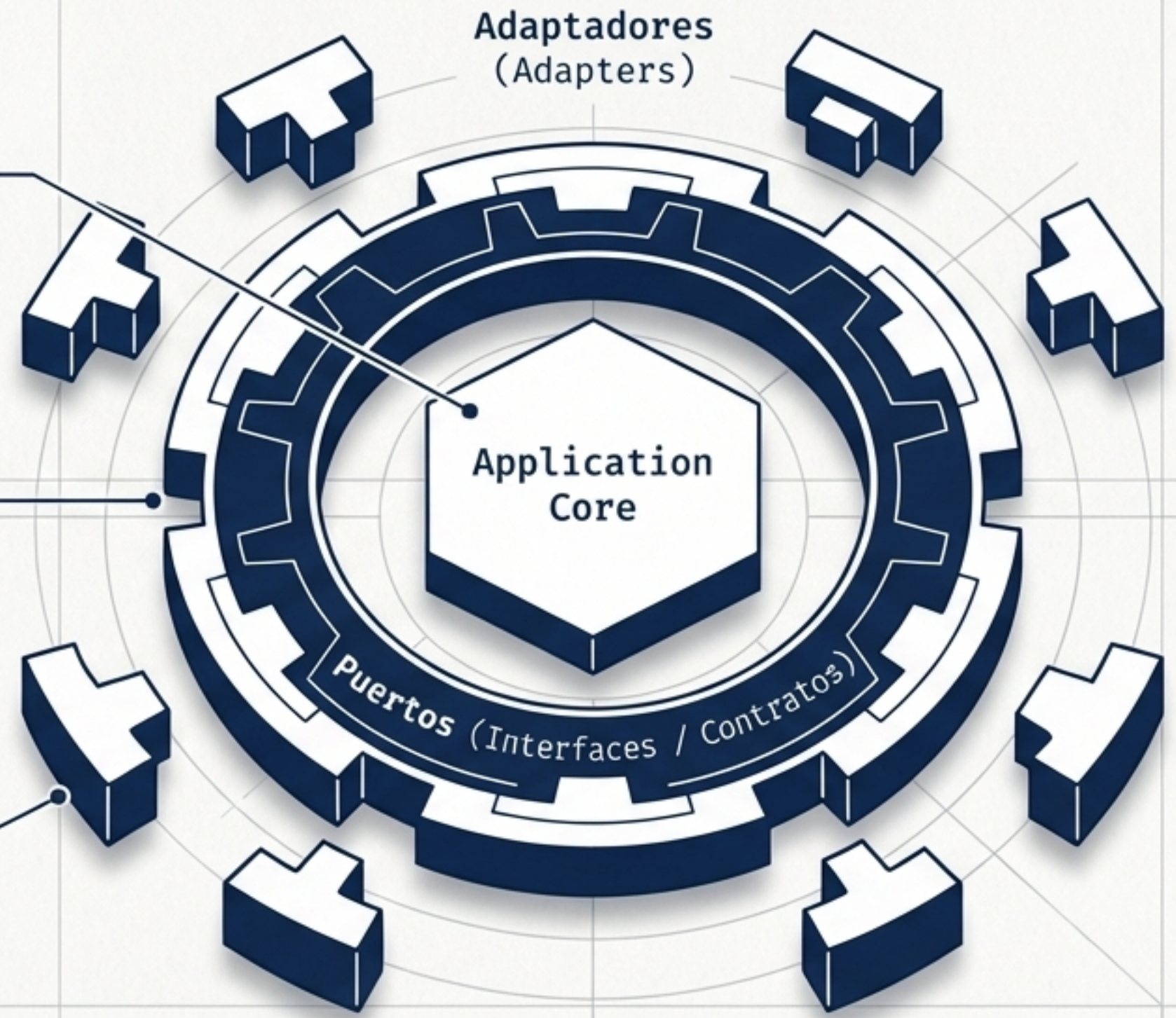
El corazón. Código puro. Contiene el Dominio y los Casos de Uso.

2. Puertos (Interfaces / Contratos)

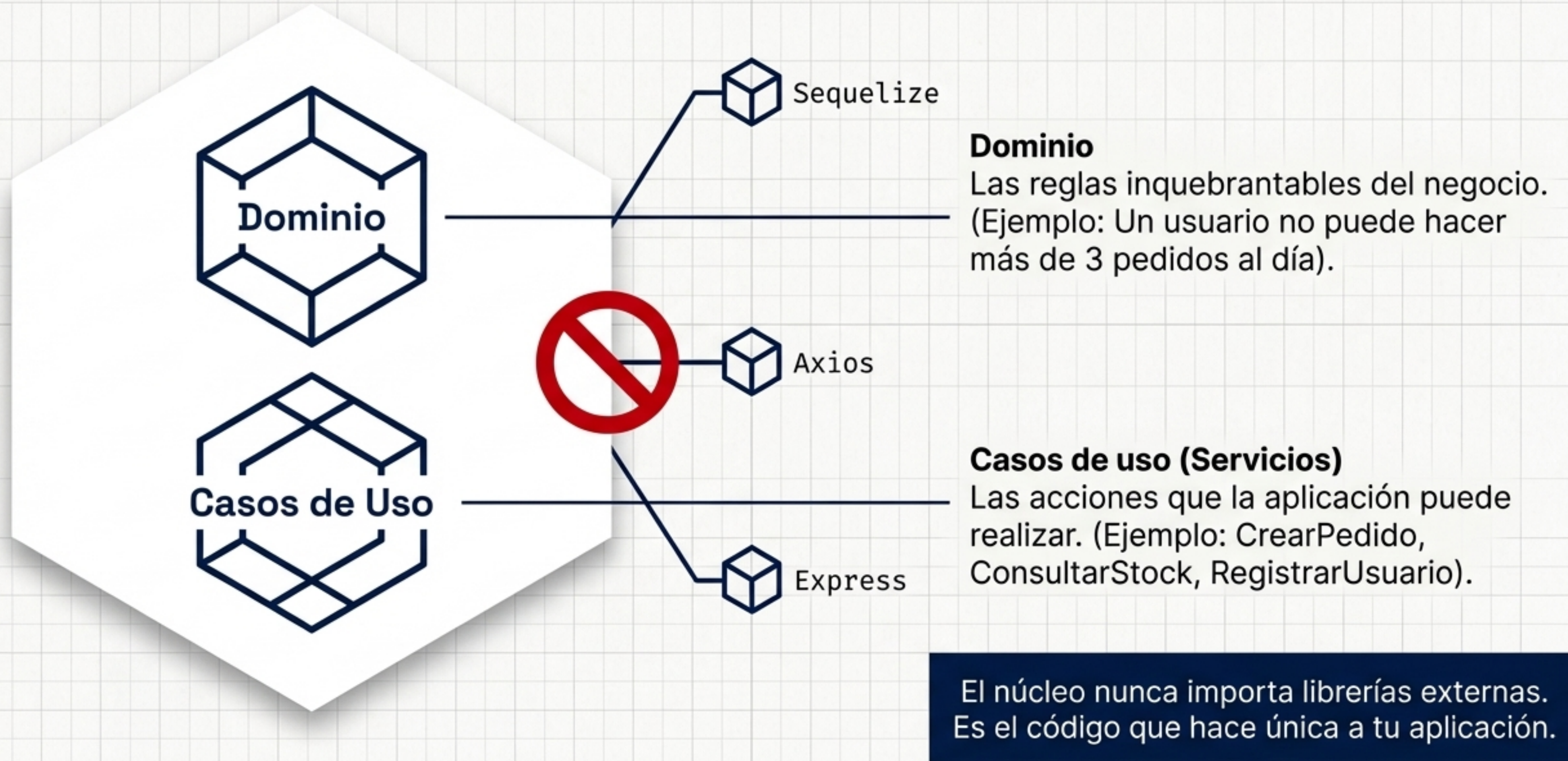
La frontera. Definen cómo se comunica el núcleo con el exterior sin contener código real.

3. Adaptadores (Adapters)

El mundo exterior. Traducen y ejecutan las implementaciones concretas.



1. El Núcleo: Código Puro y Reglas de Negocio



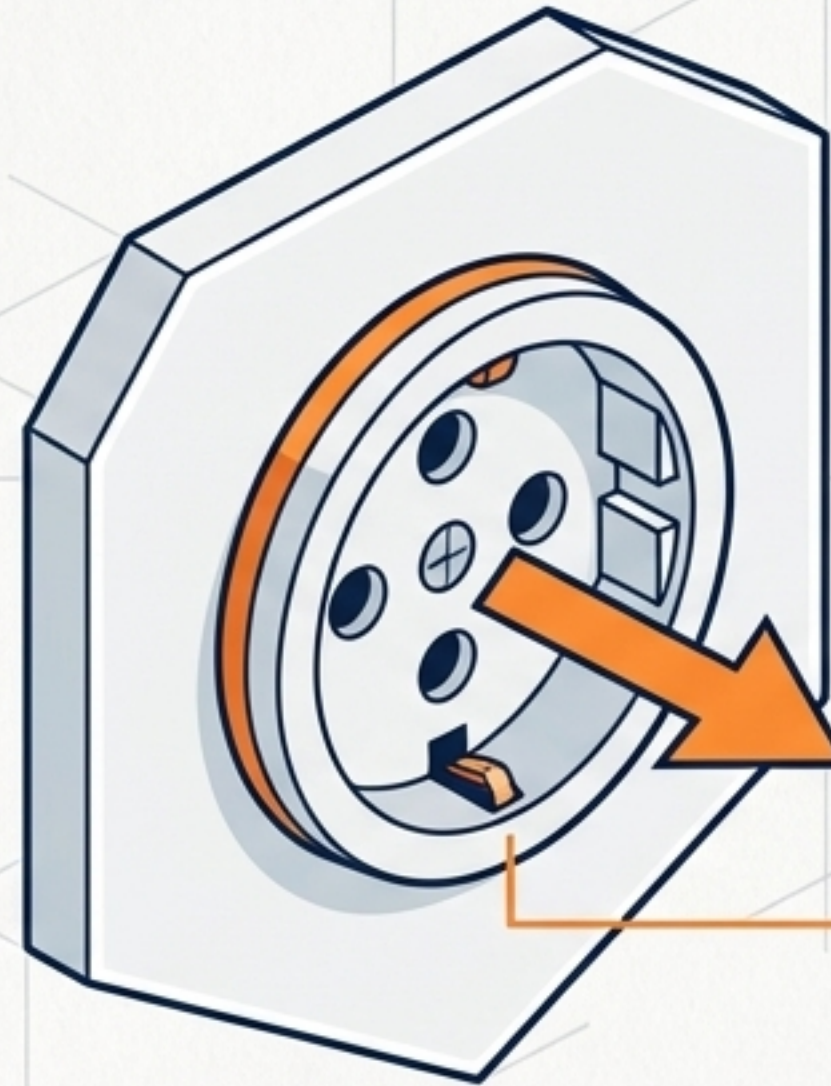
2. Puertos: Los Enchufes de la Aplicación

Un puerto es un contrato o interfaz. No contiene código real, solo la firma de los métodos. Define la **forma estándar de conectarse**, sin importar qué aparato enchufes.



Puerto de Entrada (Driver Port)

Define cómo el exterior llama a la app. (Ejemplo: la interfaz PedidoService con crearPedido(datos)).



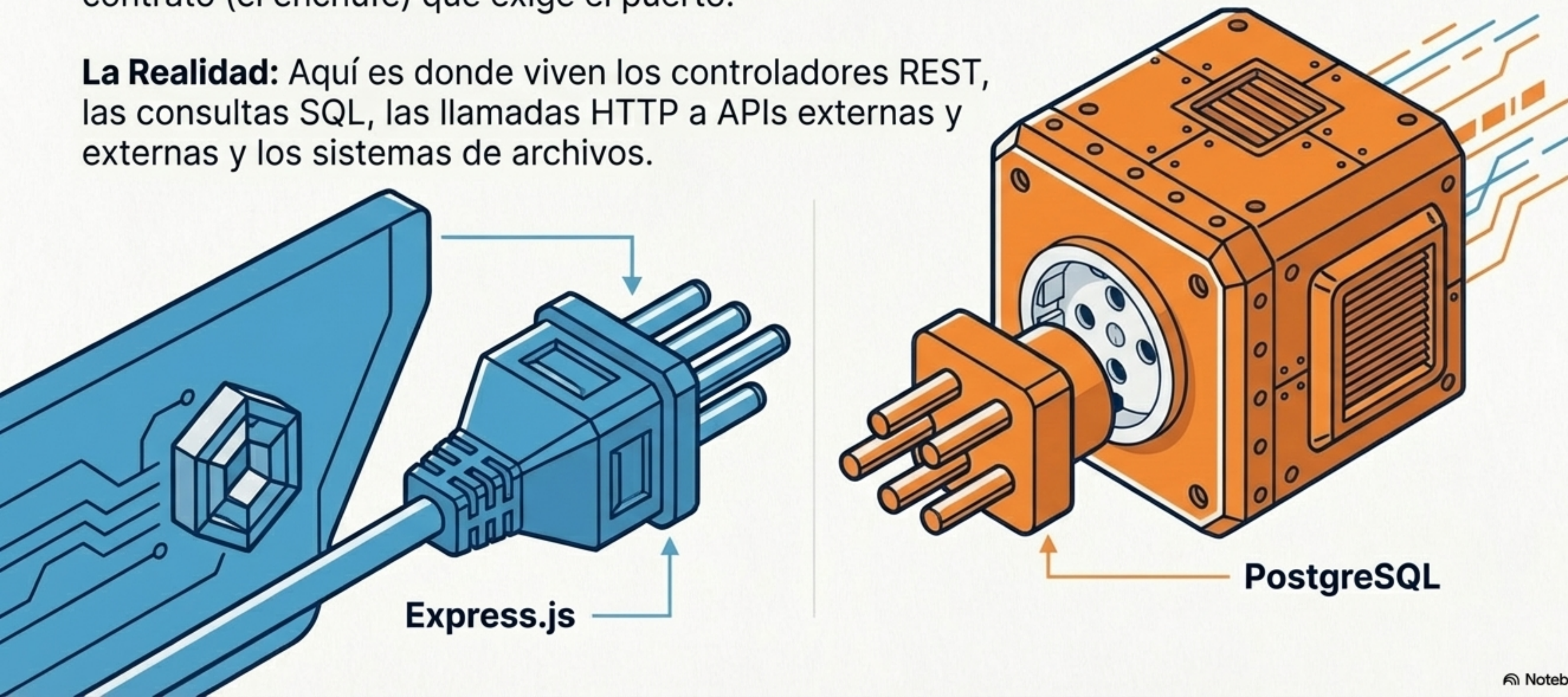
Puerto de Salida (Driven Port)

Define cómo la app llama al exterior. (Ejemplo: la interfaz PedidoRepository con guardar(pedido)).

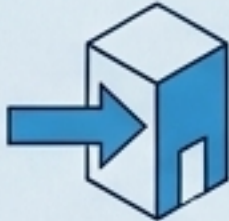
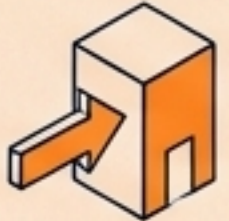
3. Adaptadores: Las Implementaciones Concretas

Son traductores físicos entre el mundo exterior y el núcleo. Cumplen exactamente con el contrato (el enchufe) que exige el puerto.

La Realidad: Aquí es donde viven los controladores REST, las consultas SQL, las llamadas HTTP a APIs externas y externas y los sistemas de archivos.



Adaptadores Primarios vs. Secundarios

	 Primarios (Fuentes de ENTRADA)	 Secundarios (Fuentes de SALIDA)
Rol Técnico	Reciben peticiones del exterior y las pasan al núcleo.	Reciben llamadas del núcleo y las ejecutan en el exterior.
Término Técnico (Ejemplos)	Controlador REST / API Web, CLI (Línea de comandos), UI Web / Móvil, Messaging Subscriber.	Base de datos SQL/NoSQL, API externa, Messaging Publisher, Sistema de archivos.
Ejemplo Real	Un endpoint POST /pedidos en Express o un botón Confirmar compra.	Un PostgresUserRepository que hace INSERT INTO o una llamada a la API de Stripe.

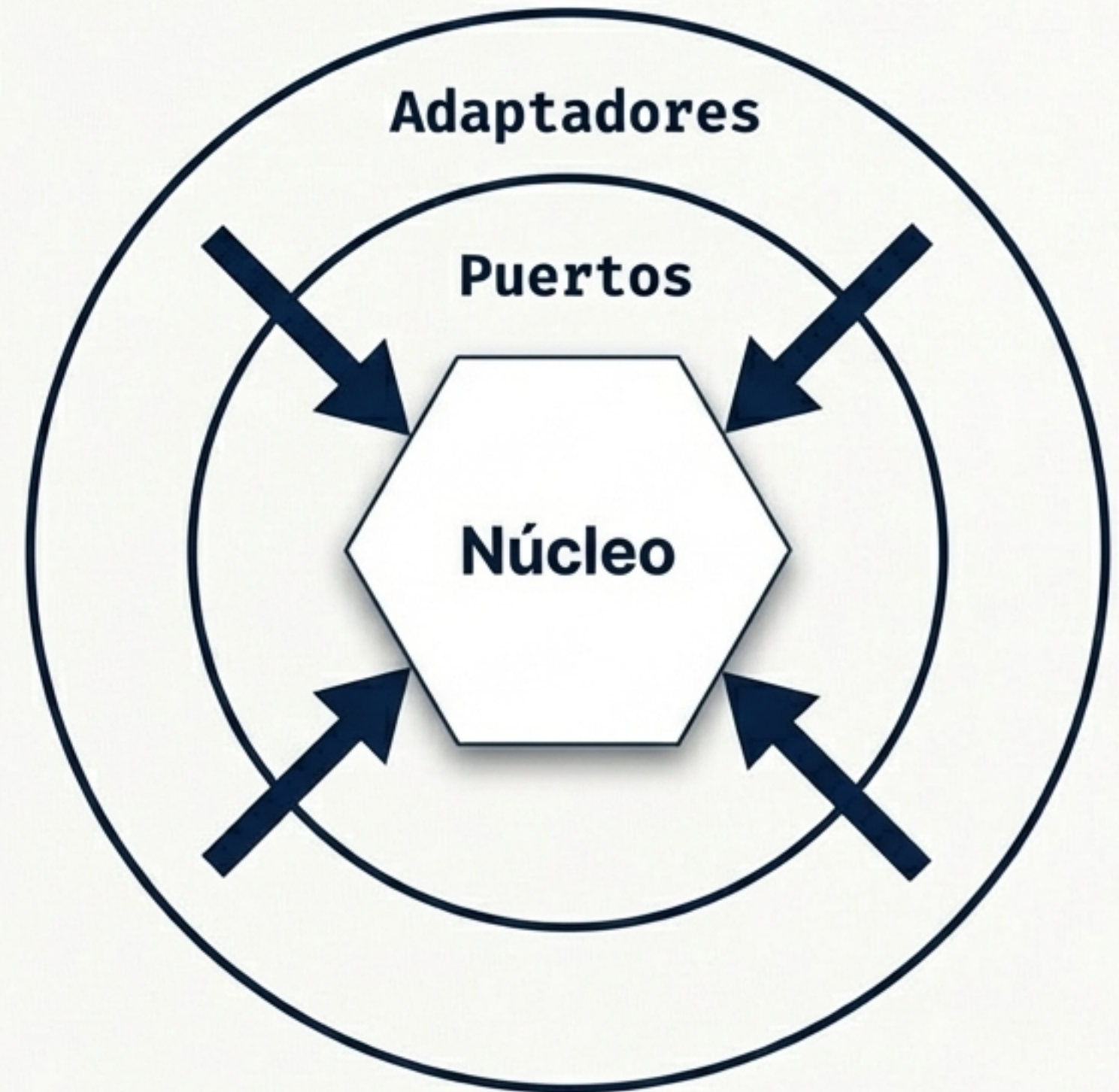
La Regla Inquebrantable de Dependencias

Todas las dependencias apuntan hacia adentro.

Los **Adaptadores** dependen de los **Puertos**.

Los **Puertos** pertenecen al **Núcleo**.

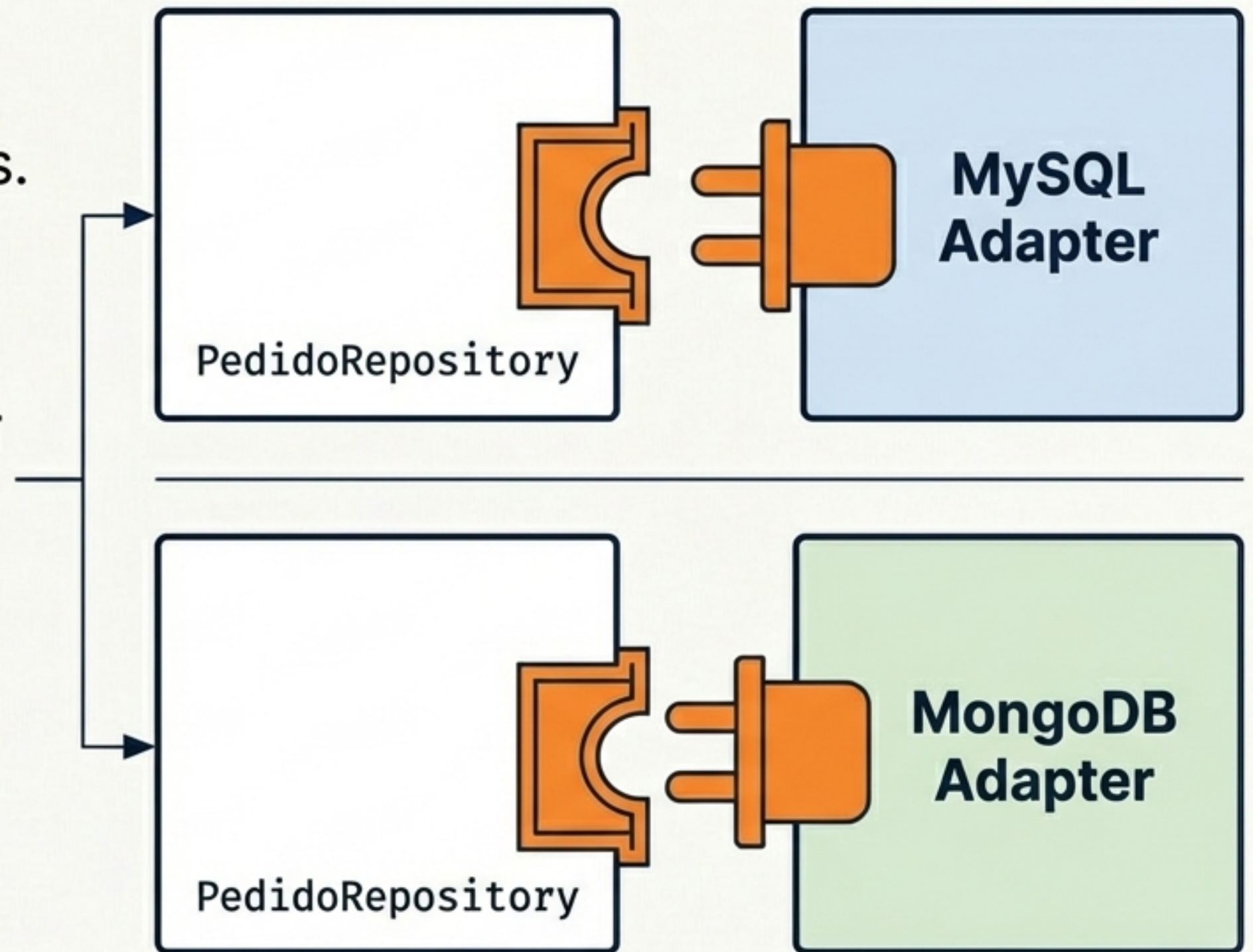
El **Núcleo** no depende de nada.
Él define las reglas, el exterior las obedece.



El Poder del Contrato

El núcleo solo conoce **PedidoRepository**. No sabe (ni le importa) cuál base de datos utilizas.

Cambiar de una base de datos relacional a una NoSQL es solo cuestión de enchufar un adaptador nuevo mediante inyección de dependencias. El **código de negocio** se mantiene **intacto**.



Flujo Completo: El Viaje de Hacer Pedido



El Retorno de Inversión Arquitectónica



Testeabilidad

Puedes probar toda la lógica de negocio usando repositorios en memoria. Cero necesidad de levantar una base de datos real.



Intercambiabilidad

Cambiar de MySQL a MongoDB solo requiere escribir un nuevo adaptador; el núcleo de la aplicación no se toca.



Independencia de Framework

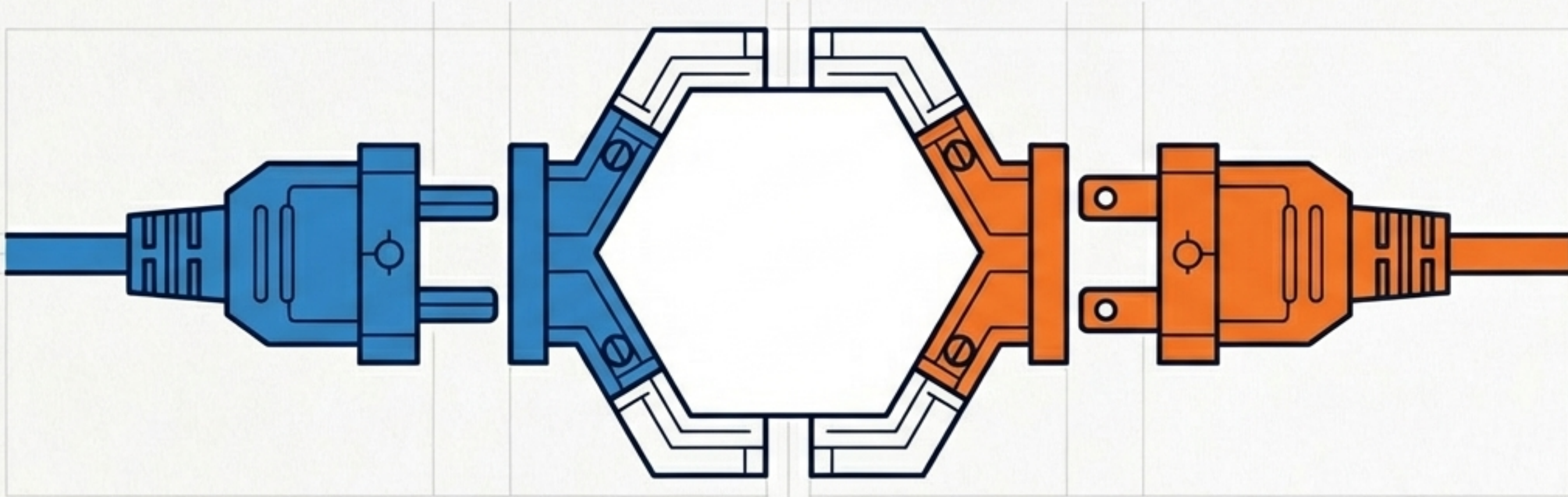
Puedes migrar tu servidor web de Express a Fastify sin modificar ni una sola regla de negocio.



Longevidad

Las tecnologías mueren, el dominio permanece. El núcleo (lo más valioso) queda blindado contra la obsolescencia tecnológica.

En Resumen



El dominio define los contratos. El exterior los cumple. El núcleo nunca sabe con quién habla.